Stage 1 · Validation

Draw clips from the detections, verify them, and measure how often the classifier is right.

STAGE 1 - VALIDATION DATA

Selection step `steps 1–2`

1 **Read classifier outputs** `read_selection_tables()`
stacks the BirdNET selection tables: every prediction with its score

2 **Select clips** `select_clips(sampling_mode = …)`
one of three designs; the same night of 12 hourly recordings under each

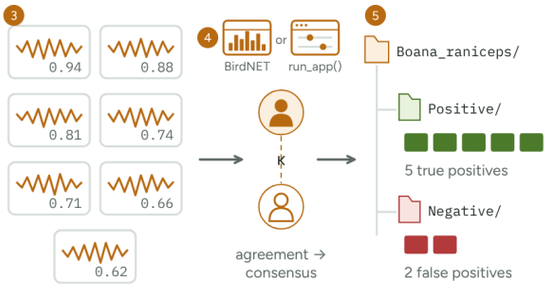


! `list_recordings()` needed by random_recordings: lists every recording, including those without predictions

! optional `covariate_bins()`
stratifies the random_detections draw along a gradient (e.g. temperature)

STAGE 1 - REVIEW APP

Review step `steps 3–5`



3 **Organize clips** `cut_clips() · pack_clips()`
cuts each clip from its recording; one zip per species

4 **Verification** `BirdNET GUI or run_app()`
several reviewers (see below): ***assign_reviewers()***, ***split_by_reviewer()***, ***reviewer_agreement()*** (κ), ***consensus_verdicts()***

5 **Read verdicts** `classify_review() or app_verdicts()`
Positive/Negative folders (GUI) or the app → one verdict per clip;
optional: ***nightly_presence()*** gives confirmed presence by night

`extract_validation_clips()` runs steps 1 → 2 → 3 in one call

`run_app()` covers steps 2–5 in a browser app

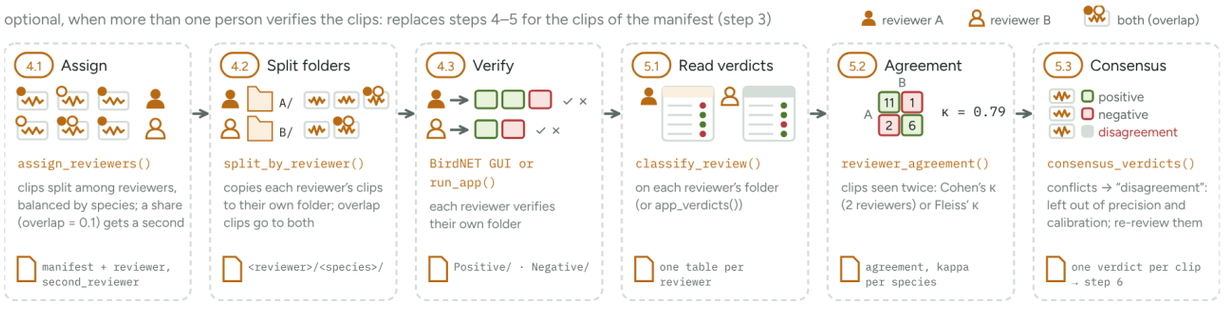
`post_validation()` runs steps 5 and 6.2–6.5; reads the verdicts itself, so it can replace step 5

STAGE 1 - SEVERAL REVIEWERS

Review step · several reviewers `optional, steps 4.1–5.3`

optional, when more than one person verifies the clips: replaces steps 4–5 for the clips of the manifest (step 3)

reviewer A · reviewer B · both (overlap)



Summary step `steps 6.1–6.5`

input: validation_by_clip.csv from step 5 (classify_review() or app_verdicts()), one verdict per clip



`review_support()` 7A: calls suggest_thresholds() (6.3) again and adds the diagnosis

Diagnostic step `step 7: 7A or 7B`

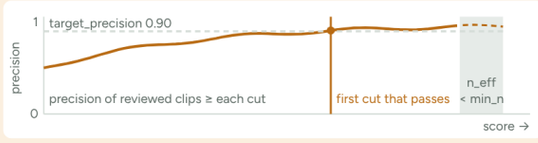
7 **What did the verification give?** two questions, one per route; answer one, or both to compare them

7A Empirical threshold

"What is the threshold for 90% of precision in my dataset?"

how review_support() (it calls suggest_thresholds())

gives a threshold per site × species, and a diagnosis:
reached · review more · exhausted · unreliable · not reviewed

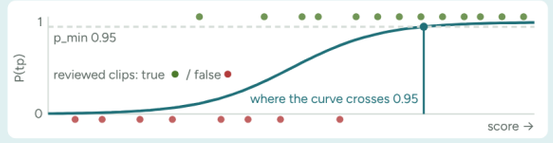


7B Model-based threshold

"What is the threshold for a 95% probability of true positive?"

how calibrate() (Stage 2 model, checked) → calibrated_thresholds()

gives a threshold per site × species from the fitted curve, and calibration_quality(): ok · poor · insufficient



compare 7A and 7B: agreement supports the model · the workflow of each is in the figure "Stage 1 · Diagnostic (7A / 7B)"

SHEEPDOG

Stage 1 · Diagnostic (7A / 7B)

Step 7 of Figure 1 in detail: A, the empirical threshold and its diagnosis (7A); B, the model-based threshold and its check (7B); and how to compare them.

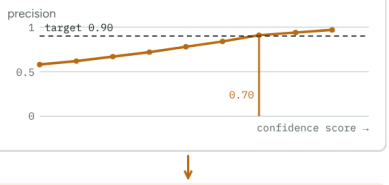
A · Empirical threshold

7A in Figure 1 · "What is the threshold for 90% of precision in my dataset?"

Get the threshold `A · step 1`

`review_support()`
target_precision 0.90 · min_n 15
conf_high 0.7 · min_precision 0.5

`inside: suggest_thresholds()`
precision of the reviewed clips at or above each cut, weighted by design



per site × species: the lowest cut that passes (≥ min_n clips), and a diagnosis

threshold only where diagnosis = reached

Read the diagnosis and act on it `A · step 2`

review_support() asks, in this order, one label per site × species; each label says what to do next

- any reviewed clip in the cell? (`n_reviewed = 0`) — no → **not reviewed** → draw clips for this cell ↻ Figure 1 · step 2
- yes →
- did suggest_thresholds() find a cut? (`precision ≥ target, n_eff ≥ min_n`) — yes → **reached** → the threshold can be used: go to step 3
- no →
- fewer than min_n high-score detections left? (`unreviewed_high < 15`) — yes → **exhausted *** → keep the clips: the model (B) may still give a threshold, or leave the cell out
- no →
- precision at high scores below min_precision? (`prec_high < 0.5`) — yes → **unreliable *** → the classifier is the limit: retrain it, then a new round ↻ Figure 1 · step 2
- no →
- **review more** → new random_detections round (same threshold, new seed) ↻ Figure 1 · step 2; stack all rounds (6.1), then A again

high-score clips are left and precision is not yet bad

high score = confidence ≥ conf_high (0.7)

* retraining may help: see Stage 1 · Retraining

Enough data to continue? `A · step 3`

- ☐ every cell you need is "reached" (or left out on purpose)
- ☐ the threshold rests on ≥ min_n clips (n_eff) above the cut
- ☐ for occupancy: a validation design at the same threshold

all met ↓

Community matrices
detection history built by hand from the threshold (score ≥ cut, k detections per night), then community_matrices() (Stage 3)

Occupancy with false positives
export_occupancy_data(), then in crywolf: fp_read_sheepdog() → fp_aggregate(threshold = cut); crywolf estimates the false positives

not yet → back to step 2 · no model: the threshold holds for the set of kept detections, not for each one (see B for the model-based route)

B · Model-based threshold

7B in Figure 1 · "What is the threshold for a 95% probability of true positive?"

Fit the calibration `B · step 1`

`calibrate()`
fits the Stage 2 model to the random_detections clips (step 6)

held-out log-loss (lower is better)



m0 one curve · m1 + species intercept
m2 + species slope · m3 + site · m4 species × site

writes model_comparison.csv, the check (step 2) and thresholds.csv; engine = "brms" for the Bayesian fit

Check it `B · step 2`

`calibration_quality()`
per species, on held-out clips

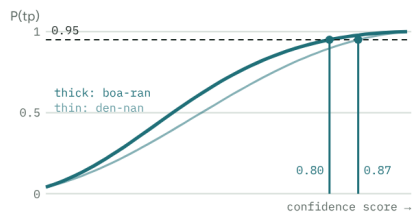
species	gain	AUC	slope	status
boa-ran	+.21	.91	1.02	ok
den-nan	+.12	.84	0.93	ok
sci-nas	+.01	.58	0.41	poor
pse-pla	n = 9	–	–	insufficient

gain: beats the species' base rate
AUC: ranks true clips above false
slope ≈ 1: probabilities are right

check before using the curve: only species marked ok go on to step 3

Get the threshold `B · step 3`

`calibrated_thresholds()`
fit, target = 0.95: the score where each species' curve reaches 0.95



one threshold per species (or site × species), for species ok in step 2; rule = "precision" asks the A question; brms adds threshold_lower

Act on the check `B · step 4`

- **ok** → use the threshold from step 3, then step 5 (what comes next)
- **poor *** → add a covariate (month, site), or use the empirical threshold (A) for this species
- **insufficient** → too few true or false clips: review more ↻ Figure 1 · step 2; calibration_effort() says how many

* if neither helps, see Retraining

B gives no review_support() diagnosis; later, cell_status() (Stage 3) labels each cell reliable, probably absent or unreliable

What comes next `B · step 5`

species marked ok in step 2 · the routes are compared in "Routes to detection histories and occupancy"

Community matrices (Stage 3)

way 1	`detection_history(rule = "mean")` P(tp) ≥ 0.95 under the fitted curve	lme4 or brms
way 2	`detection_history(rule = "probability",` prob_min = 0.9): cautious where review was thin	brms
way 3	`history_draws()` → `community_matrices_draws()` one history per draw: mean and interval per cell	brms

then cell_status() → suggest_core_species() → community_matrices() (frequency, counts, effort, trials) → vegan or gllvm

Occupancy with false positives (crywolf)

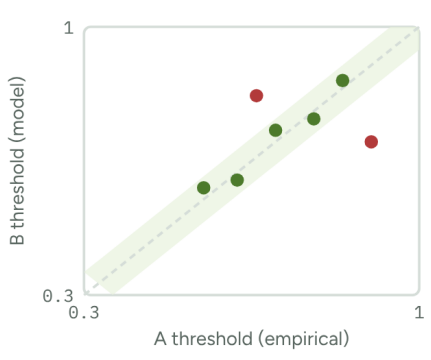
threshold	`calibrated_thresholds()` (or `threshold_lower`) → `export_occupancy_data()` without fit → `fp_aggregate(threshold = score)`	lme4 or brms
draws	`export_occupancy_data(fit = brms fit)` → `fp_draws_data(rule = "sample")` → `fp_occu_mc()`: one fit per draw, pooled	brms

- ☐ a random validation design at the same threshold (random_detections) and/or random_recordings
- ☐ a calibration without a site effect, or local prevalence is counted twice

Compare A and B `after both`

Compare the two thresholds

same cells: A against B run with rule = "precision", target = 0.90 (like with like)



they agree
the model reproduces what the reviewed clips show: report Stage 2 (way 1–3); A stays as the check

they differ in some cells
B higher: pooling may lend the cell a curve; A higher: precision may dip above the first cut. Look at those cells (A step 2, B step 4)

B with P(tp) 0.95 asks a stricter question (each detection, not the set), so its thresholds usually sit higher than A at 0.90: compare with rule = "precision" to see whether the model fits

Stage 1 · Retraining

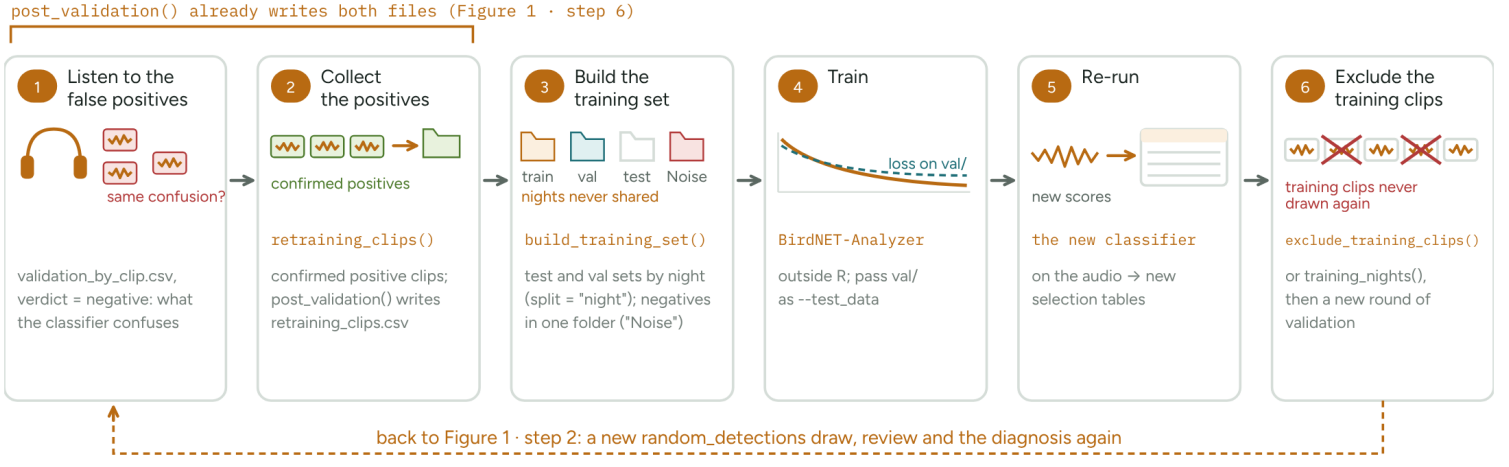
When the classifier, not the amount of review, limits precision: retrain it with your verified clips, then validate the new classifier from Figure 1 · step 2.

When to retrain `from the A and B labels`

route	label	retrain?	why, and what to try first
A	unreliable	yes	precision stays below min_precision even at high scores: more review only confirms it (Figure 2, A · step 2)
A	exhausted	consider	nothing left to review and the target is far off: the classifier caps the precision
A	review more	not yet	review another round first (Figure 1 · step 2)
B	poor	after the options	first try more clips across the scores, a covariate or another model (Figure 4 · step 3); retrain if still poor
B	insufficient	not yet	too few true or false clips: review more
A or B	one cause of FPs	yes	you listened to the false-positive clips (loop step 1) and most come from one confusing species or a recurring noise

A = empirical route (review_support() label), B = model-based route (calibration_quality() status), as in Figures 2 and 4; FPs = false positives

The retraining loop `steps 1–6`



Before and after retraining `checks`

- ☐ ≥ 100 positive clips per class
  min_files in build_training_set() warns below it (BirdNET-Analyzer recommendation)
- ☐ negatives from your reviews
  the false positives teach the classifier what to reject; put them in one non-event folder
- ☐ split by night (or by site), not by clip
  clips of the same night are correlated; the checks of build_training_set() should all be 0
- ☐ keep training_set.csv
  with the new classifier: exclude_training_clips() needs it, so test clips are never training clips
- ☐ compare old and new
  the same random_detections design on nights neither classifier was trained on (training_nights())

SHEEPDOG

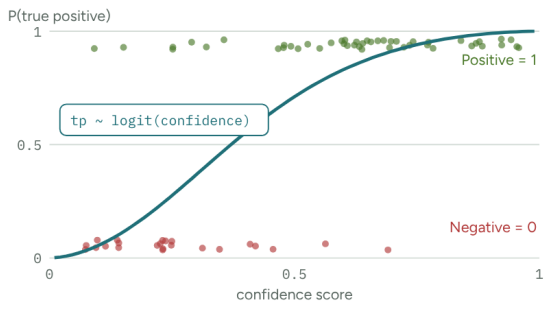
Stage 2 · Calibration

Check that the score-to-probability calibration is reliable, apply it to every prediction, and turn the probabilities into detections for further analyses.

I · IS THE CALIBRATION RELIABLE?

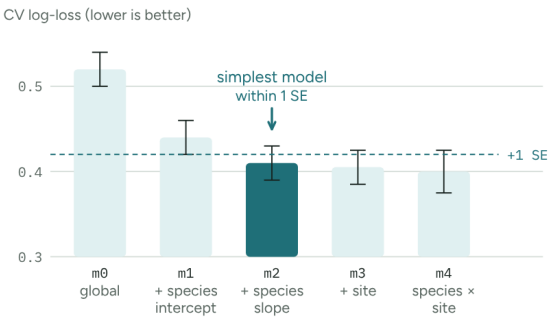
Fit the score-to-probability curve `step 1`

Input: the `random_detections` clips verified in Stage 1 (`validation_by_clip.csv`, Figure 1 · step 6), read with `read_validations()`. Each clip is 1 (true) or 0 (false) at its score; the curve is a logistic regression on logit(score), with species and site effects.



Choose the model `step 2`

`compare_calibration_models()` fits five candidates with cross-validation by night; `select_calibration_model()` keeps the simplest within one standard error of the best.



Check each species `step 3`

`calibration_quality()` on held-out clips asks: does the model beat the base rate, does it rank true clips above false ones, are its probabilities right? Only species marked ok go on.

species	adds info (log-loss gain)	discriminates (AUC)	calibrated (slope ≈ 1)	status
boa-ran	✓ +0.21	✓ 0.91	✓ 1.02	ok
den-nan	✓ +0.12	✓ 0.84	✓ 0.93	ok
sci-nas	× +0.01	× 0.58	× 0.41	poor
pse-pla	n = 9	–	–	insufficient

held-out clips, per species · calibration_quality()

what to do with each status (limits are provisional: read the reason column)

- ok → go on to step 4
- poor * → • score says little (low AUC, no gain): more random_detections clips across the scores (top clips lower the AUC), or a covariate (month, site) among the candidates
  • probabilities off (slope, intercept): another model (species slope, single-species family)
  • still poor: leave it out of P(tp) analyses, or use the empirical threshold (Figure 2, route A)
- insufficient → fewer than min_class (10) true or false clips: review more ↻ Figure 1 · step 2; calibration_effort() says how many. Meanwhile its curve is borrowed from other species and unchecked

* if nothing helps, see Stage 1 · Retraining

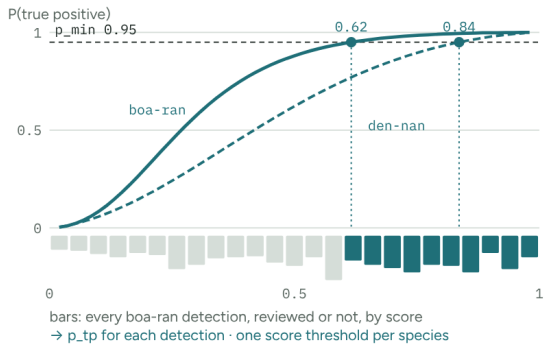
`calibrate()` runs steps 1–5 (way 1) in one call and writes their files

`engine = "brms"` the same, fitted with brms: adds step 7 and ways 2–3

II · APPLY IT AND TURN PROBABILITIES INTO DETECTIONS

Apply the curve to every detection `step 4`

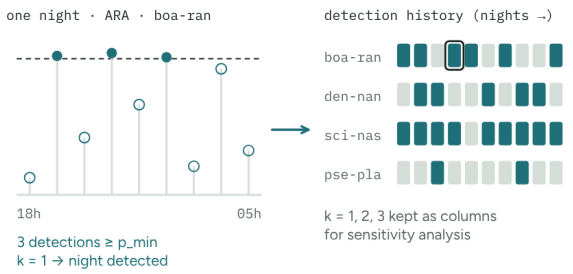
`score_detections()` gives every detection its P(tp), reviewed or not. `calibrated_thresholds()` gives the score where each curve reaches `p_min` (Wood & Kahl 2024).



bars: every boa-ran detection, reviewed or not, by score
→ p_tp for each detection · one score threshold per species

Collapse detections into nights `step 5`

`detection_history()`: a site × species × night counts as detected when at least k detections count. k = 1, 2 and 3 are kept for sensitivity.



3 detections ≥ p_min
k = 1 → night detected

k = 1, 2, 3 kept as columns for sensitivity analysis

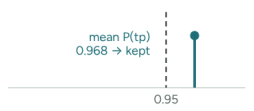
Three ways to decide which detections count `step 6`

The rule of step 5. Way 1 needs any calibration; ways 2 and 3 need the posterior of a brms fit (step 7).

`way 1` `rule = "mean"`

lme4 or brms

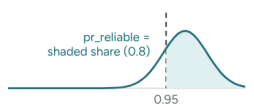
kept if P(tp) ≥ p_min (0.95)
one history; the curve is treated as exact



`way 2` `rule = "probability"`

brms

kept if pr_reliable ≥ prob_min (0.9)
one cautious history: drops detections near the cut



`way 3` `history_draws()`

brms

kept if P(tp) ≥ p_min in each draw
one history per posterior draw
→ matrices with intervals

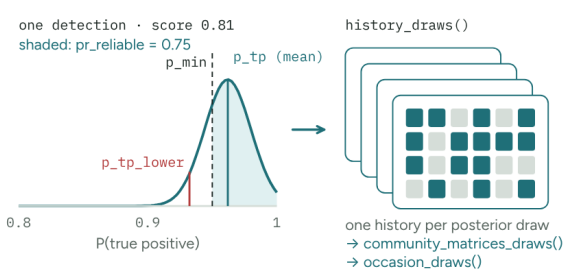


in all three, a night counts when ≥ k detections count (k = 1, 2, 3)

III · CARRY THE UNCERTAINTY AND GO ON

Fit with brms `step 7`

`engine = "brms"`: each detection gets a posterior, with the mean `p_tp`, a lower bound `p_tp_lower` and `pr_reliable`. Checks: `calibration_prior()`, `calibration_diagnostics()`, `calibration_sensitivity()`, `calibration_ppc()`.



one history per posterior draw
→ community_matrices_draws()
→ occasion_draws()

prior check · R-hat · ESS · sensitivity · pp check

What comes next `step 8`

From the history (way 1 or 2) or the histories (way 3) to community matrices, or to occupancy with false positives.

species with calibration_quality() ok · the routes are compared in "Routes to detection histories and occupancy"

Community matrices (Stage 3)

`cell_status()`
each site × species cell: reliable, probably absent or unreliable

`suggest_core_species()`
keeps species with no unreliable cell

`community_matrices()`
from the way 1 or 2 history: frequency, counts, effort, trials

`community_matrices_draws()`
from the way 3 histories: mean and interval per cell

→ `vegan` or `gllvm`
ordination (PCoA, NMDS, RDA) or gllvm with trials

Occupancy with false positives (crywolf)

`threshold` `calibrated_thresholds()` (or `threshold_lower`) → `export_occupancy_data()` without fit → `fp_aggregate(threshold = score)` — lme4 or brms

`draws` `export_occupancy_data(fit = brms fit)` → `fp_draws_data(rule = "sample")` → `fp_occu_mc()`: one fit per draw, pooled — brms

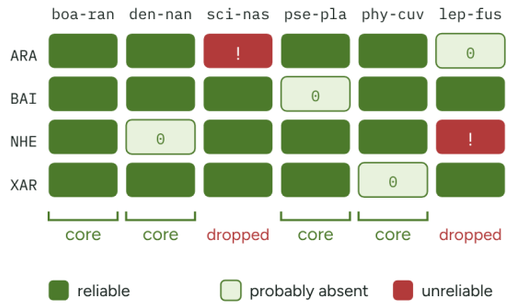
☐ a random validation design at the same threshold (random_detections) and/or random_recordings
☐ a calibration without a site effect, or local prevalence is counted twice

Stage 3 · Community matrices

Keep the reliable cells, count nights with a strict entry rule, standardise by effort and build site × month matrices for ordination; or export to an occupancy model with false positives.

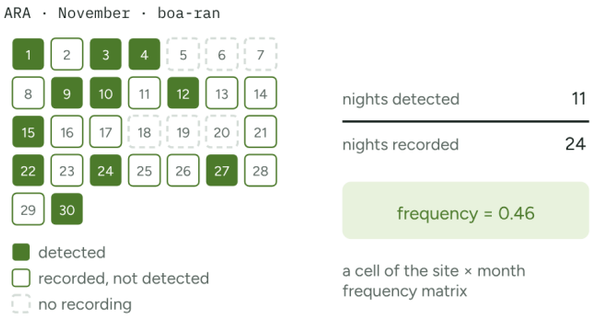
Keep the cells you can trust `steps 1–2`

`cell_status(fit, validations, p_min, min_n)` labels each site × species cell reliable (the curve reaches `p_min` within the reviewed scores, with ≥ `min_n` clips), probably absent (reviewed, no true positive: a real zero) or unreliable (a zero that could be a false negative). `suggest_core_species()` keeps the species with no unreliable cell; the final choice is yours. Needs the Stage 2 fit.



Count nights, standardise by effort `step 3`

A night counts as detected when it has at least `k_min` (2) reliable detections (P(tp) ≥ `p_min`). Nights recorded are the distinct nights of the site in the history. Site × month cells with fewer than `min_nights_month` (5) nights are dropped.



`build_community_matrices()` reads the history file and runs steps 3–4 for the core species you give

Assemble the site × month matrices `step 4`

`community_matrices()` gives frequency (nights detected ÷ nights recorded), counts, effort and trials, one column per core species. For way 3, `community_matrices_draws()` gives a mean and an interval per cell.

	boa-ran	den-nan	pse-pla	phy-cuv
ARA·Oct	0.38	0.71	0.10	0.52
ARA·Nov	0.46	0.63	0.00	0.58
ARA·Dec	0.29	0.80	0.13	0.40
BAI·Oct	0.00	0.55	0.62	0.21
BAI·Nov	0.08	0.49	0.74	0.33
BAI·Dec	0.17	0.60	0.68	0.25

+ count matrix · effort table · trials matrix (gllvm)

Analyse the community `step 5`

Ordination (PCoA, NMDS, RDA) in vegan, or gllvm with the trials matrix as binomial denominators. With way 3, repeat the analysis over draws and pool.



vegan (PCoA, NMDS, RDA) · gllvm with the trials matrix

Occupancy with false positives

A separate ending, not a step of Stage 3: export the occasions, scores and reviews, and let an occupancy model estimate the false positives (index section “Occupancy models”).

Export occasions, scores and reviews `step 1`

`export_occupancy_data()` writes every occasion of the survey, including those with no detection, with its score and, if reviewed, its verdict and sampling design, to one folder with a documented schema.

occasion	top score	review
ARA · n1 · 19h		confirmed
ARA · n1 · 21h		rejected
ARA · n2 · 20h		—
BAI · n1 · 23h		confirmed
BAI · n2 · 19h	0	—
NHE · n1 · 22h		—

```
sheepdog-occupancy/
occasions.csv
detections.csv
scores.csv
validation.csv
sites.csv
calibration_draws.parquet
schema.json
```

→ crywolf, or any software

Two routes into crywolf `step 2`

A score threshold (any route) gives one data set; the brms calibration draws give one per draw.

threshold route — lme4 or brms

score threshold: Stage 1 (A) or calibrated_thresholds() (B)

```
export_occupancy_data() without fit
crywolf: fp_read_sheepdog() → fp_aggregate(threshold = )
→ fp_data() → fp_occu(): one fit
```

draws route — brms

the calibration itself: one detection matrix per draw

```
export_occupancy_data(fit = brms fit)
crywolf: fp_draws_data(rule = "sample" or "threshold")
→ fp_occu_mc(): one fit per draw, pooled
```

both need

☐ known truth: random_detections at the same threshold (observation confirmation) and/or random_recordings (multiple methods)
☐ a calibration without a site effect, or local prevalence is counted twice

crywolf estimates p10 itself from the known truth: see the crywolf explainer for how it works and how the designs differ